Lesson 03 Demo 07

Creating EKS Cluster Using CloudFormation Template

Objective: To automate the provisioning of cloud resources using the CloudFormation service by creating a template for launching a new EKS Cluster

Tools required: AWS CloudFormation

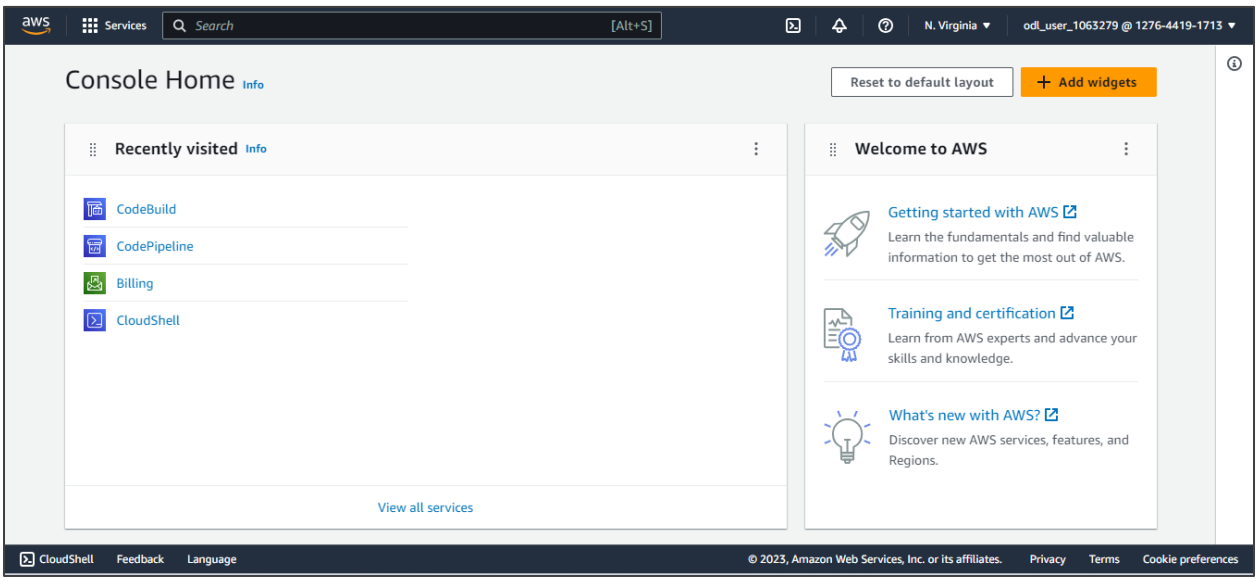
Prerequisites: None

Steps to be followed:

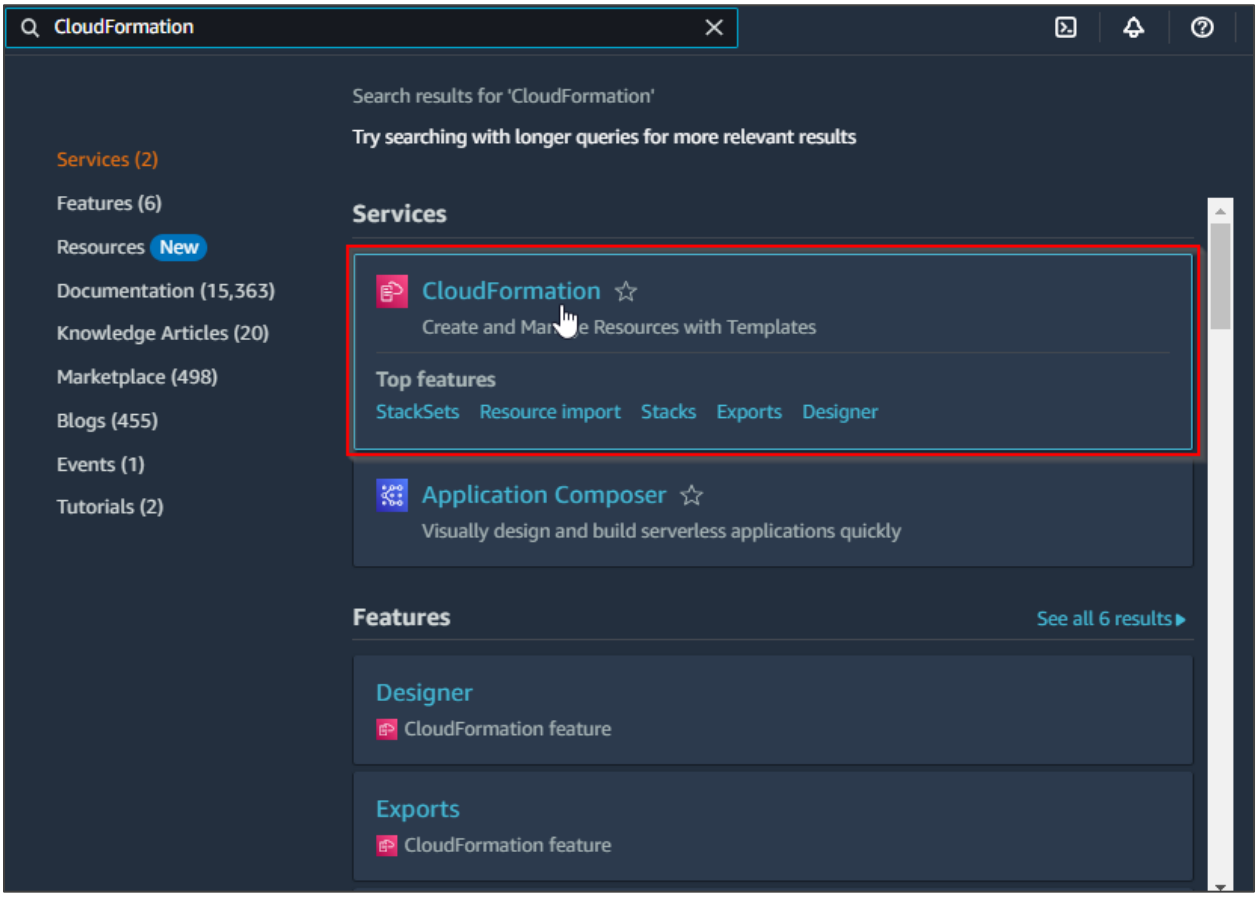
1. Create a new CloudFormation template to launch a new VM
2. Deploy and validate the EKS Cluster

Step 1: Create a new CloudFormation template to launch a new VM

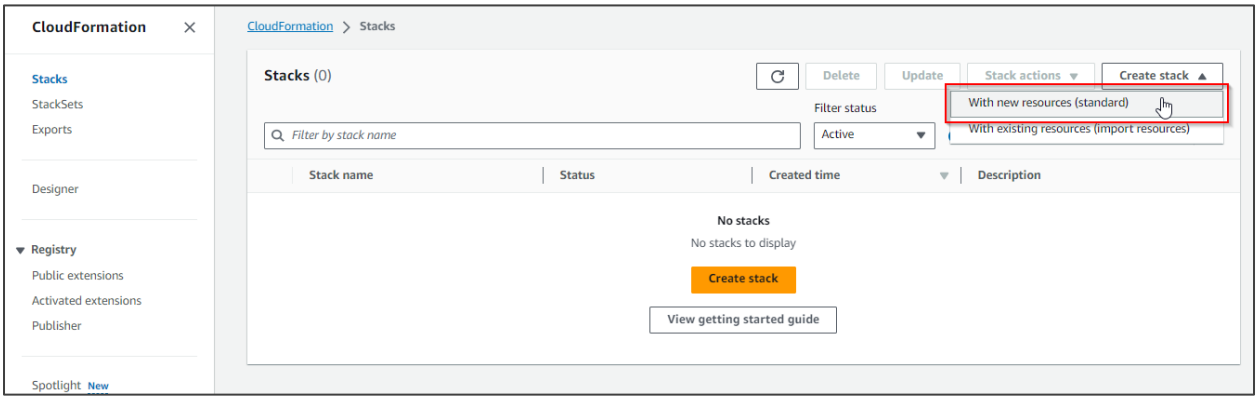
1.1 Sign in to the AWS management console



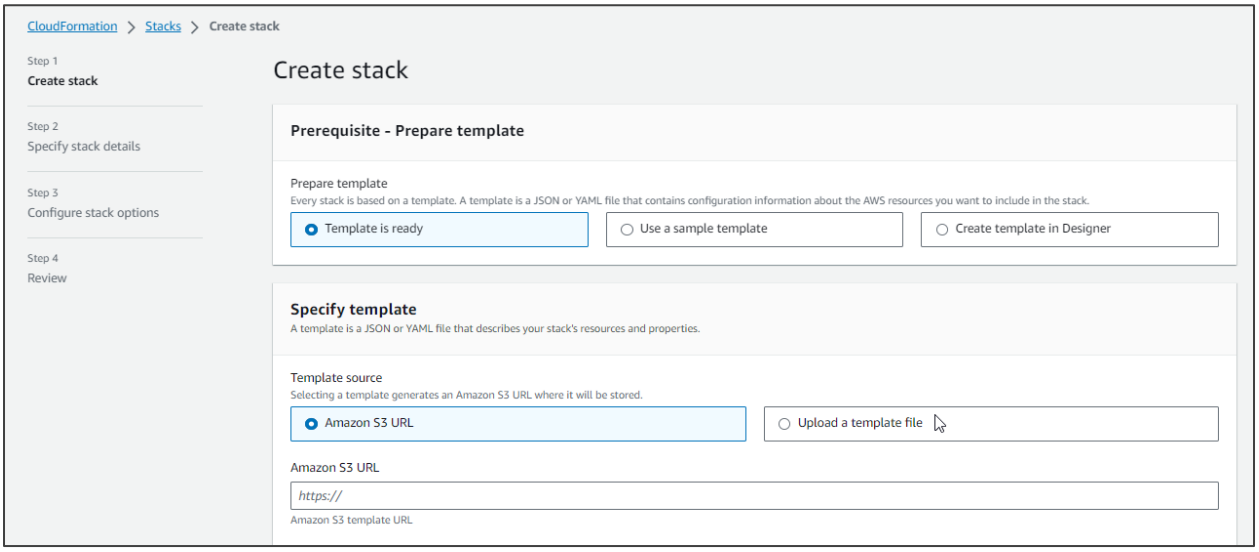
1.2 Search for and click on **CloudFormation**



1.3 Click on the **Create stack** button and select the **With new resources (standard)** option



You will see the following interface:

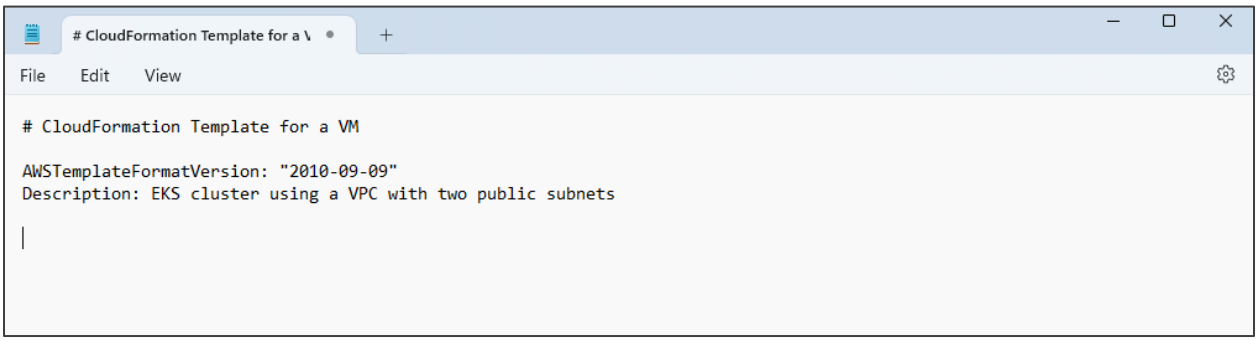


1.4 Create the CloudFormation template before adding it to the CloudFormation service

Note: Create the CloudFormation template using Notepad and include the following header as shown in the screenshot below:

 # CloudFormation Template for a VM

AWSTemplateFormatVersion: "2010-09-09"
Description: EKS cluster using a VPC with two public subnets



1.5 Add all required parameters for the CloudFormation template, either taking input from the user or hard coding them as defaults

```
Parameters:
  EKSClusterName:
    Type: String
    Description: Name of k8s cluster
    Default: eks-cluster
  NumWorkerNodes:
    Type: Number
    Description: Number of worker nodes to create
    Default: 2
  WorkerNodesInstanceType:
    Type: String
    Description: EC2 instance type for the worker nodes
    Default: t3.medium
  KeyPairName:
    Type: String
    Description: Name of an existing EC2 key pair (for SSH-access to the worker node
instances)
    Default: eksKey
Mappings:
  VpcIpRanges:
    Option1:
      VPC: 10.100.0.0/16
      PublicSubnet1 : 10.100.0.0/20
      PublicSubnet2: 10.100.16.0/20
      PrivateSubnet1: 10.100.32.0/20
      PrivateSubnet2: 10.100.48.0/20
      PersistenceSubnet1: 10.100.64.0/20
      PersistenceSubnet2: 10.100.80.0/20
  EksAmiIds:
    us-east-1:
      Standard: ami-0164b8ae1906d3372
```

```
# CloudFormation Template for a \
File   Edit   View
AWSTemplateFormatVersion: "2010-09-09"
Description: EKS cluster using a VPC with two public subnets

Parameters:
  EKSClusterName:
    Type: String
    Description: Name of k8s cluster
    Default: eks-cluster
  NumWorkerNodes:
    Type: Number
    Description: Number of worker nodes to create
    Default: 2
  WorkerNodesInstanceType:
    Type: String
    Description: EC2 instance type for the worker nodes
    Default: t3.medium
  KeyPairName:
    Type: String
    Description: Name of an existing EC2 key pair (for SSH-access to the worker node instances)
    Default: eksKey
Mappings:
  VpcIpRanges:
    Option1:
      VPC: 10.100.0.0/16
      PublicSubnet1 : 10.100.0.0/20
      PublicSubnet2: 10.100.16.0/20
      PrivateSubnet1: 10.100.32.0/20
      PrivateSubnet2: 10.100.48.0/20
      PersistenceSubnet1: 10.100.64.0/20
      PersistenceSubnet2: 10.100.80.0/20
  EksAmiIds:
    us-east-1:
      Standard: ami-0164b8ae1906d3372

Ln 36, Col 1        100%        Windows (CRLF)
```

1.6 Once parameters are created, add a **Resources** section to create the VPC, Subnets, Security Groups, EKS Cluster, and other resources

```
Resources:

#=================================================================
=====#
 # VPC

#=================================================================
=====#

  VPC:
    Type: AWS::EC2::VPC
    Properties:
      CidrBlock: !FindInMap [ VpcIpRanges, Option1, VPC ]
      EnableDnsSupport: true
      EnableDnsHostnames: true
      Tags:
        - Key: Name
          Value: !Ref AWS::StackName
  PublicSubnet1:
    Type: AWS::EC2::Subnet
```

```
  Properties:
    VpcId: !Ref VPC
    CidrBlock: !FindInMap [ VpcIpRanges, Option1, PublicSubnet1 ]
    AvailabilityZone: !Select [ 0, !GetAZs '' ]
    Tags:
      - Key: Name
        Value: !Sub "${AWS::StackName}-PublicSubnet1"
      - Key: kubernetes.io/role/elb
        Value: 1
      - Key: !Sub "kubernetes.io/cluster/${AWS::StackName}"
        Value: shared
PublicSubnet2:
  Type: AWS::EC2::Subnet
  Properties:
    VpcId: !Ref VPC
    CidrBlock: !FindInMap [ VpcIpRanges, Option1, PublicSubnet2 ]
    AvailabilityZone: !Select [ 1, !GetAZs '' ]
    Tags:
      - Key: Name
        Value: !Sub "${AWS::StackName}-PublicSubnet2"
      - Key: kubernetes.io/role/elb
        Value: 1
      - Key: !Sub "kubernetes.io/cluster/${AWS::StackName}"
        Value: shared
PrivateSubnet1:
  Type: AWS::EC2::Subnet
  Properties:
    VpcId: !Ref VPC
    CidrBlock: !FindInMap [ VpcIpRanges, Option1, PrivateSubnet1 ]
    AvailabilityZone: !Select [ 0, !GetAZs '' ]
    Tags:
      - Key: Name
        Value: !Sub "${AWS::StackName}-PrivateSubnet1"
      - Key: kubernetes.io/role/internal-elb
        Value: 1
      - Key: !Sub "kubernetes.io/cluster/${AWS::StackName}"
        Value: shared
PrivateSubnet2:
  Type: AWS::EC2::Subnet
  Properties:
    VpcId: !Ref VPC
    CidrBlock: !FindInMap [ VpcIpRanges, Option1, PrivateSubnet2 ]
    AvailabilityZone: !Select [ 1, !GetAZs '' ]
```

```
    Tags:
      - Key: Name
        Value: !Sub "${AWS::StackName}-PrivateSubnet2"
      - Key: kubernetes.io/role/internal-elb
        Value: 1
      - Key: !Sub "kubernetes.io/cluster/${AWS::StackName}"
        Value: shared
PersistenceSubnet1:
  Type: AWS::EC2::Subnet
  Properties:
    VpcId: !Ref VPC
    CidrBlock: !FindInMap [ VpcIpRanges, Option1, PersistenceSubnet1 ]
    AvailabilityZone: !Select [ 0, !GetAZs '' ]
    Tags:
      - Key: Name
        Value: !Sub "${AWS::StackName}-PersistenceSubnet1"
PersistenceSubnet2:
  Type: AWS::EC2::Subnet
  Properties:
    VpcId: !Ref VPC
    CidrBlock: !FindInMap [ VpcIpRanges, Option1, PersistenceSubnet2 ]
    AvailabilityZone: !Select [ 1, !GetAZs '' ]
    Tags:
      - Key: Name
        Value: !Sub "${AWS::StackName}-PersistenceSubnet2"
InternetGateway:
  Type: AWS::EC2::InternetGateway
  Properties:
    Tags:
      - Key: Name
        Value: !Ref AWS::StackName
VPCGatewayAttachment:
  Type: AWS::EC2::VPCGatewayAttachment
  Properties:
    InternetGatewayId: !Ref InternetGateway
    VpcId: !Ref VPC
RouteTable:
  Type: AWS::EC2::RouteTable
  Properties:
    VpcId: !Ref VPC
    Tags:
      - Key: Name
        Value: !Sub "${AWS::StackName}-PublicSubnets"
```

```
InternetGatewayRoute:
  Type: AWS::EC2::Route
  DependsOn: VPCGatewayAttachment
  Properties:
    RouteTableId: !Ref RouteTable
    DestinationCidrBlock: 0.0.0.0/0
    GatewayId: !Ref InternetGateway
NatGateway1EIP:
  Type: AWS::EC2::EIP
  DependsOn: VPCGatewayAttachment
  Properties:
    Domain: vpc
NatGateway2EIP:
  Type: AWS::EC2::EIP
  DependsOn: VPCGatewayAttachment
  Properties:
    Domain: vpc
NatGateway1:
  Type: AWS::EC2::NatGateway
  Properties:
    AllocationId: !GetAtt NatGateway1EIP.AllocationId
    SubnetId: !Ref PublicSubnet1
NatGateway2:
  Type: AWS::EC2::NatGateway
  Properties:
    AllocationId: !GetAtt NatGateway2EIP.AllocationId
    SubnetId: !Ref PublicSubnet2
PrivateRouteTable1:
  Type: AWS::EC2::RouteTable
  Properties:
    VpcId: !Ref VPC
    Tags:
      - Key: Name
        Value: !Sub ${AWS::StackName} Private Routes (AZ1)
DefaultPrivateRoute1:
  Type: AWS::EC2::Route
  Properties:
    RouteTableId: !Ref PrivateRouteTable1
    DestinationCidrBlock: 0.0.0.0/0
    NatGatewayId: !Ref NatGateway1
PrivateRouteTable2:
  Type: AWS::EC2::RouteTable
  Properties:
```

```
    VpcId: !Ref VPC
    Tags:
      - Key: Name
        Value: !Sub ${AWS::StackName} Private Routes (AZ1)
DefaultPrivateRoute2:
  Type: AWS::EC2::Route
  Properties:
    RouteTableId: !Ref PrivateRouteTable2
    DestinationCidrBlock: 0.0.0.0/0
    NatGatewayId: !Ref NatGateway2

PublicSubnet1RouteTableAssociation:
  Type: AWS::EC2::SubnetRouteTableAssociation
  Properties:
    SubnetId: !Ref PublicSubnet1
    RouteTableId: !Ref RouteTable
PublicSubnet2RouteTableAssociation:
  Type: AWS::EC2::SubnetRouteTableAssociation
  Properties:
    SubnetId: !Ref PublicSubnet2
    RouteTableId: !Ref RouteTable
PrivateSubnet1RouteTableAssociation:
  Type: AWS::EC2::SubnetRouteTableAssociation
  Properties:
    SubnetId: !Ref PrivateSubnet1
    RouteTableId: !Ref PrivateRouteTable1
PrivateSubnet2RouteTableAssociation:
  Type: AWS::EC2::SubnetRouteTableAssociation
  Properties:
    SubnetId: !Ref PrivateSubnet2
    RouteTableId: !Ref PrivateRouteTable2
PersistenceSubnet1RouteTableAssociation:
  Type: AWS::EC2::SubnetRouteTableAssociation
  Properties:
    SubnetId: !Ref PersistenceSubnet1
    RouteTableId: !Ref RouteTable
PersistenceSubnet2RouteTableAssociation:
  Type: AWS::EC2::SubnetRouteTableAssociation
  Properties:
    SubnetId: !Ref PersistenceSubnet2
    RouteTableId: !Ref RouteTable
```

```
#==================================================================
=====#
 # Control plane

#==================================================================
=====#

 ControlPlane:
  Type: AWS::EKS::Cluster
  Properties:
   Name: !Ref AWS::StackName
   Version: "1.27"
   RoleArn: !GetAtt ControlPlaneRole.Arn
   ResourcesVpcConfig:
    SecurityGroupIds:
     - !Ref ControlPlaneSecurityGroup
    SubnetIds:
     - !Ref PrivateSubnet1
     - !Ref PrivateSubnet2
 ControlPlaneRole:
  Type: AWS::IAM::Role
  Properties:
   AssumeRolePolicyDocument:
    Version: '2012-10-17'
    Statement:
      Effect: Allow
      Principal:
       Service:
        - eks.amazonaws.com
      Action: sts:AssumeRole
   ManagedPolicyArns:
    - arn:aws:iam::aws:policy/AmazonEKSClusterPolicy
    - arn:aws:iam::aws:policy/AmazonEKSServicePolicy


#==================================================================
=====#
 # Control plane security group

#==================================================================
=====#

 ControlPlaneSecurityGroup:
```

```
    Type: AWS::EC2::SecurityGroup
    Properties:
      GroupDescription: Security group for the elastic network interfaces between the
control plane and the worker nodes
      VpcId: !Ref VPC
      Tags:
        - Key: Name
          Value: !Sub "${AWS::StackName}-ControlPlaneSecurityGroup"

  ControlPlaneIngressFromWorkerNodesHttps:
    Type: AWS::EC2::SecurityGroupIngress
    Properties:
      Description: Allow incoming HTTPS traffic (TCP/443) from worker nodes (for API
server)
      GroupId: !Ref ControlPlaneSecurityGroup
      SourceSecurityGroupId: !Ref WorkerNodesSecurityGroup
      IpProtocol: tcp
      ToPort: 443
      FromPort: 443
  ControlPlaneEgressToWorkerNodesKubelet:
    Type: AWS::EC2::SecurityGroupEgress
    Properties:
      Description: Allow outgoing kubelet traffic (TCP/10250) to worker nodes
      GroupId: !Ref ControlPlaneSecurityGroup
      DestinationSecurityGroupId: !Ref WorkerNodesSecurityGroup
      IpProtocol: tcp
      FromPort: 10250
      ToPort: 10250
  ControlPlaneEgressToWorkerNodesHttps:
    Type: AWS::EC2::SecurityGroupEgress
    Properties:
      Description: Allow outgoing HTTPS traffic (TCP/442) to worker nodes (for pods
running extension API servers)
      GroupId: !Ref ControlPlaneSecurityGroup
      DestinationSecurityGroupId: !Ref WorkerNodesSecurityGroup
      IpProtocol: tcp
      FromPort: 443
      ToPort: 443


#==================================================================
=====#
  # Worker nodes security group
```

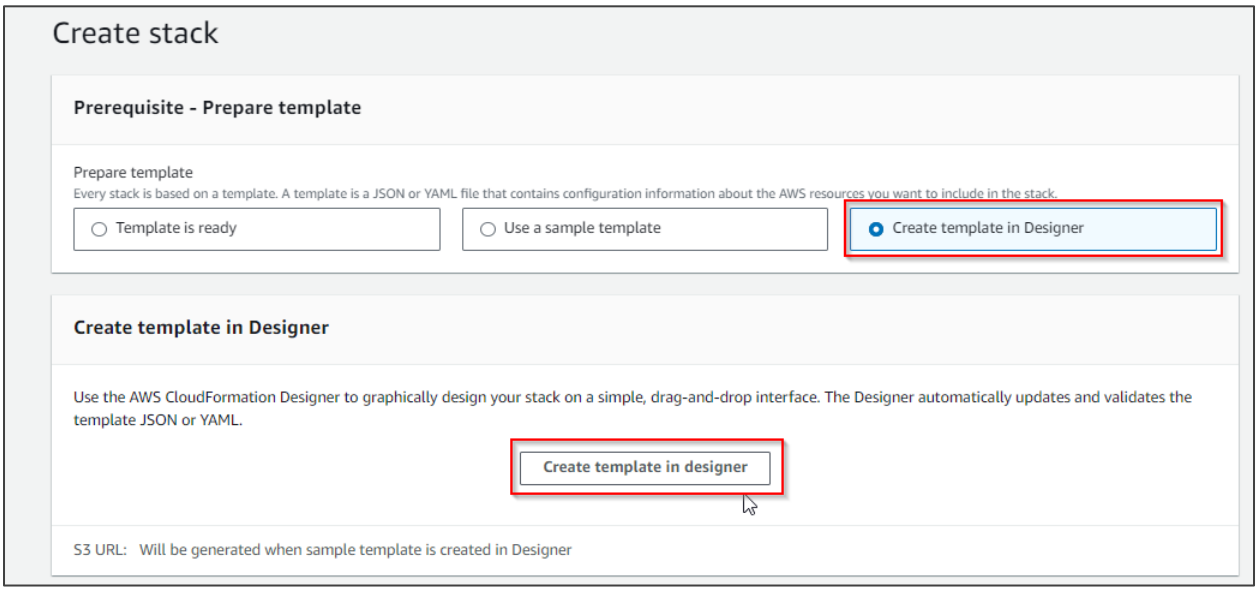
```
  # Note: Default egress rule (allow all traffic to all destinations) applies

#=========================================================================
=====#

  WorkerNodesSecurityGroup:
    Type: AWS::EC2::SecurityGroup
    Properties:
      GroupDescription: Security group for all the worker nodes
      VpcId: !Ref VPC
      Tags:
        - Key: Name
          Value: !Sub "${AWS::StackName}-WorkerNodesSecurityGroup"
        - Key: !Sub "kubernetes.io/cluster/${ControlPlane}"
          Value: "owned"
  WorkerNodesIngressFromWorkerNodes:
    Type: AWS::EC2::SecurityGroupIngress
    Properties:
      Description: Allow all incoming traffic from other worker nodes
      GroupId: !Ref WorkerNodesSecurityGroup
      SourceSecurityGroupId: !Ref WorkerNodesSecurityGroup
      IpProtocol: "-1"
  WorkerNodesIngressFromControlPlaneKubelet:
    Type: AWS::EC2::SecurityGroupIngress
    Properties:
      Description: Allow incoming kubelet traffic (TCP/10250) from control plane
      GroupId: !Ref WorkerNodesSecurityGroup
      SourceSecurityGroupId: !Ref ControlPlaneSecurityGroup
      IpProtocol: tcp
      FromPort: 10250
      ToPort: 10250
  WorkerNodesIngressFromControlPlaneHttps:
    Type: AWS::EC2::SecurityGroupIngress
    Properties:
      Description: Allow incoming HTTPS traffic (TCP/443) from control plane (for pods
running extension API servers)
      GroupId: !Ref WorkerNodesSecurityGroup
      SourceSecurityGroupId: !Ref ControlPlaneSecurityGroup
      IpProtocol: tcp
      FromPort: 443
      ToPort: 443
```

```
#=================================================================
=====#
 # Worker nodes (auto-scaling group)

#=================================================================
=====#

  WorkerNodesAutoScalingGroup:
    Type: AWS::AutoScaling::AutoScalingGroup
    UpdatePolicy:
      AutoScalingRollingUpdate:
        MinInstancesInService: 1
        MaxBatchSize: 1
    Properties:
      LaunchConfigurationName: !Ref WorkerNodesLaunchConfiguration
      MinSize: !Ref NumWorkerNodes
      MaxSize: !Ref NumWorkerNodes
      VPCZoneIdentifier:
        - !Ref PrivateSubnet1
        - !Ref PrivateSubnet2
      Tags:
        - Key: Name
          Value: !Sub "${AWS::StackName}-WorkerNodesAutoScalingGroup"
          PropagateAtLaunch: true
        # Without this tag, worker nodes are unable to join the cluster:
        - Key: !Sub "kubernetes.io/cluster/${ControlPlane}"
          Value: "owned"
          PropagateAtLaunch: true
  WorkerNodesRole:
    Type: AWS::IAM::Role
    Properties:
      AssumeRolePolicyDocument:
        Version: '2012-10-17'
        Statement:
          Effect: Allow
          Principal:
            Service:
              - ec2.amazonaws.com
          Action: sts:AssumeRole
      ManagedPolicyArns:
        - arn:aws:iam::aws:policy/AmazonEKSWorkerNodePolicy
        - arn:aws:iam::aws:policy/AmazonEKS_CNI_Policy
        - arn:aws:iam::aws:policy/AmazonEC2ContainerRegistryReadOnly
```

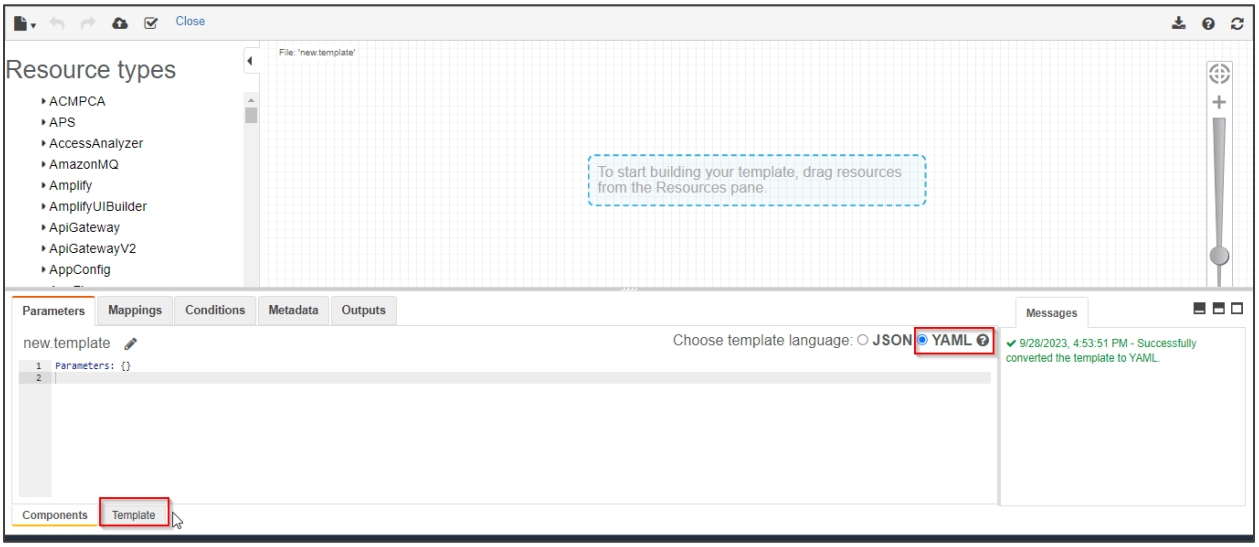
```
 # NOTE: You have to define NodeGroup (type: AWS::EKS::Nodegroup), without this
no woker nodes will be attach to cluster

 WorkerNodegroup:
  Type: AWS::EKS::Nodegroup
  DependsOn: ControlPlane
  Properties:
    ClusterName: !Sub "${AWS::StackName}"
    NodeRole: !GetAtt WorkerNodesRole.Arn
    ScalingConfig:
      MinSize:
        Ref: NumWorkerNodes
      DesiredSize:
        Ref: NumWorkerNodes
      MaxSize:
        Ref: NumWorkerNodes
    Subnets:
      - !Ref PrivateSubnet1
      - !Ref PrivateSubnet2

 WorkerNodesLaunchConfiguration:
  Type: AWS::AutoScaling::LaunchConfiguration
  # Wait until cluster is ready before launching worker nodes
  DependsOn: ControlPlane
  Properties:
    AssociatePublicIpAddress: false
    IamInstanceProfile: !Ref WorkerNodesInstanceProfile
    ImageId: !FindInMap
      - EksAmiIds
      - !Ref AWS::Region
      - Standard
    InstanceType: !Ref WorkerNodesInstanceType
    KeyName: !Ref KeyPairName
    SecurityGroups:
      - !Ref WorkerNodesSecurityGroup
 WorkerNodesInstanceProfile:
  Type: AWS::IAM::InstanceProfile
  Properties:
    Roles:
      - !Ref WorkerNodesRole
```

```
# CloudFormation Template for a \
File    Edit    View
      NodeRole: !GetAtt WorkerNodesRole.Arn
      ScalingConfig:
        MinSize:
          Ref: NumWorkerNodes
        DesiredSize:
          Ref: NumWorkerNodes
        MaxSize:
          Ref: NumWorkerNodes
      Subnets:
        - !Ref PrivateSubnet1
        - !Ref PrivateSubnet2

  WorkerNodesLaunchConfiguration:
    Type: AWS::AutoScaling::LaunchConfiguration
    # Wait until cluster is ready before launching worker nodes
    DependsOn: ControlPlane
    Properties:
      AssociatePublicIpAddress: false
      IamInstanceProfile: !Ref WorkerNodesInstanceProfile
      ImageId: !FindInMap
        - EksAmiIds
        - !Ref AWS::Region
        - Standard
      InstanceType: !Ref WorkerNodesInstanceType
      KeyName: !Ref KeyPairName
      SecurityGroups:
        - !Ref WorkerNodesSecurityGroup
  WorkerNodesInstanceProfile:
    Type: AWS::IAM::InstanceProfile
    Properties:
      Roles:
        - !Ref WorkerNodesRole
```

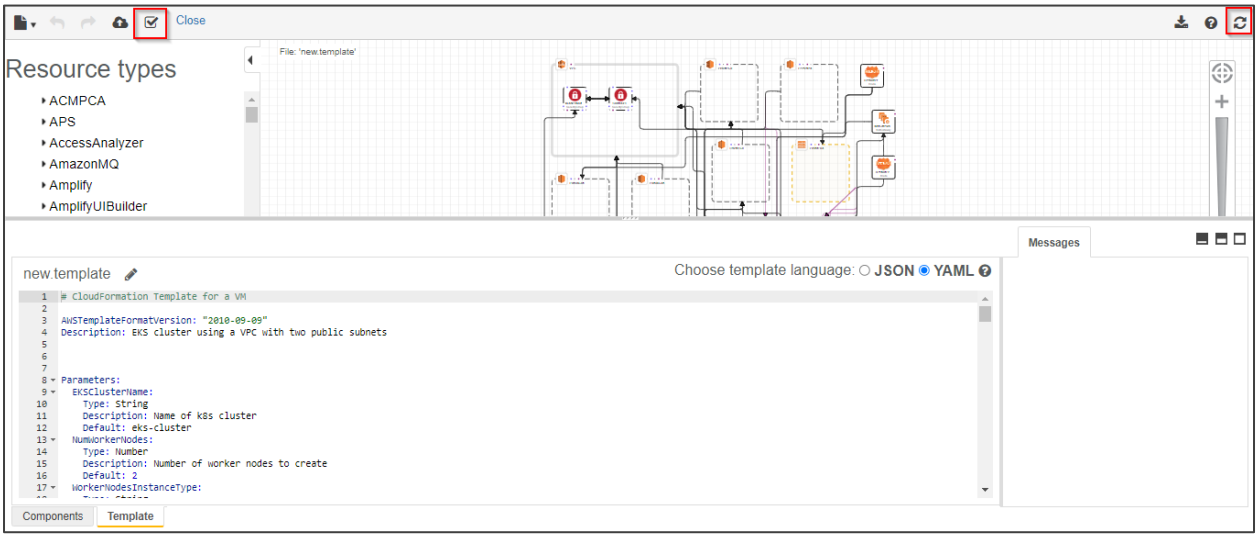
1.7 Select the **Create template in Designer** option, and click on the **Create template in designer** button



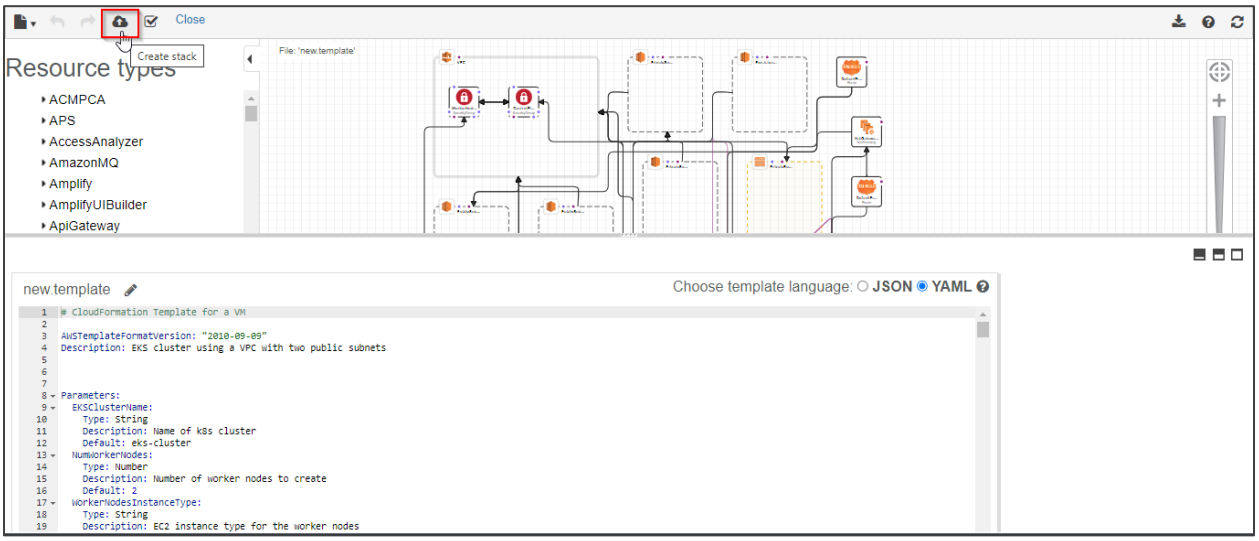
1.8 Select the **YAML** option for the template language, and on the bottom left side, click on **Template**



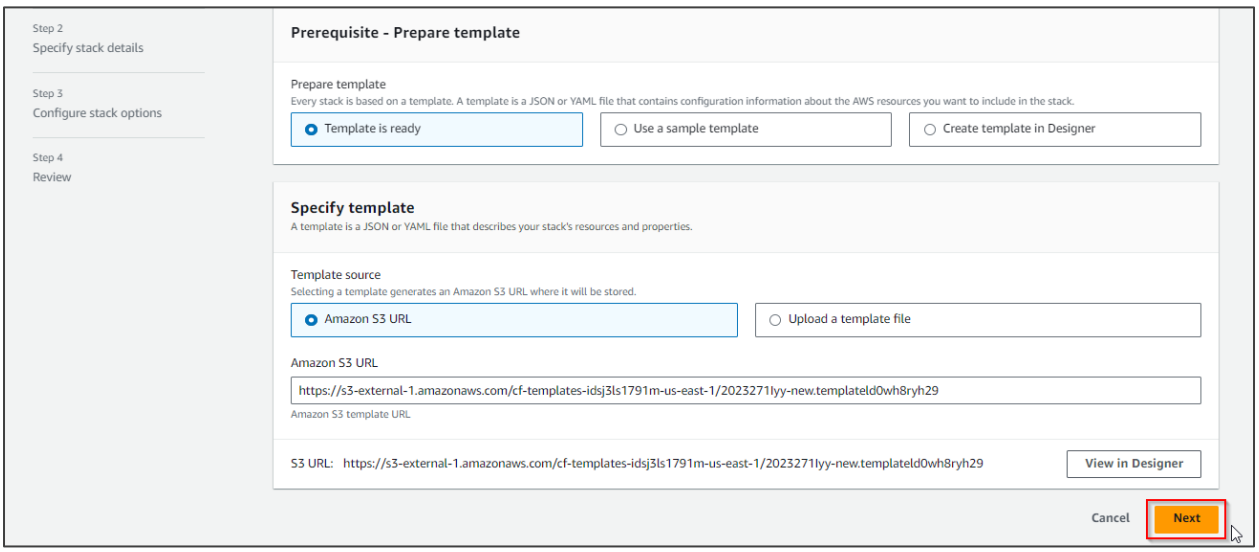
1.9 Remove the existing code from the template, enter the **YAML code** from the notepad, and click on the **refresh** and **checkmark** icons to validate the changes, as shown in the screenshot below:



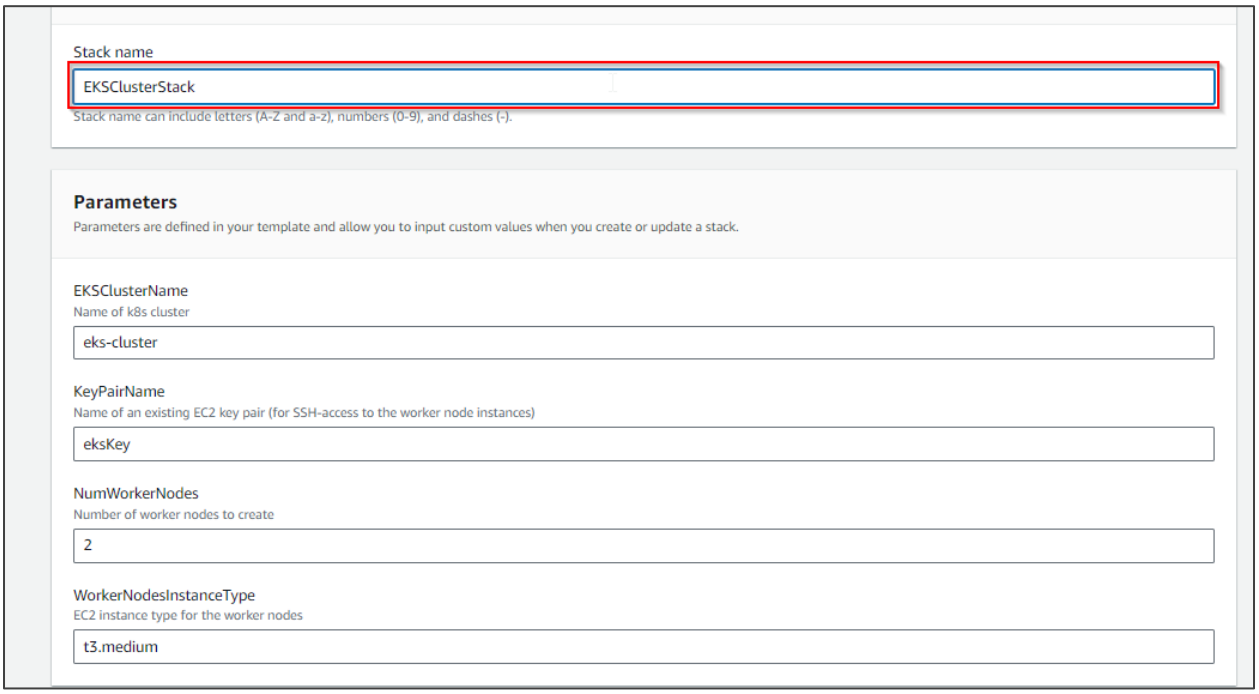
1.10 Once the validation is complete, click on the **Create stack** icon



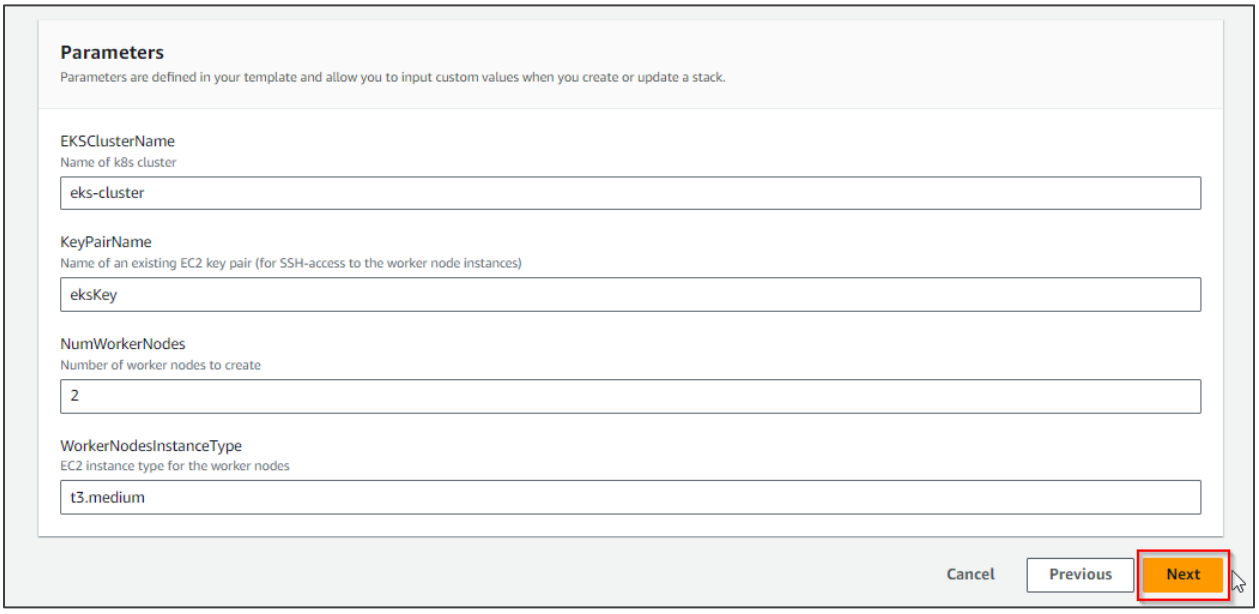
1.11 Select the template as **Amazon S3 URL** and click on the **Next** button



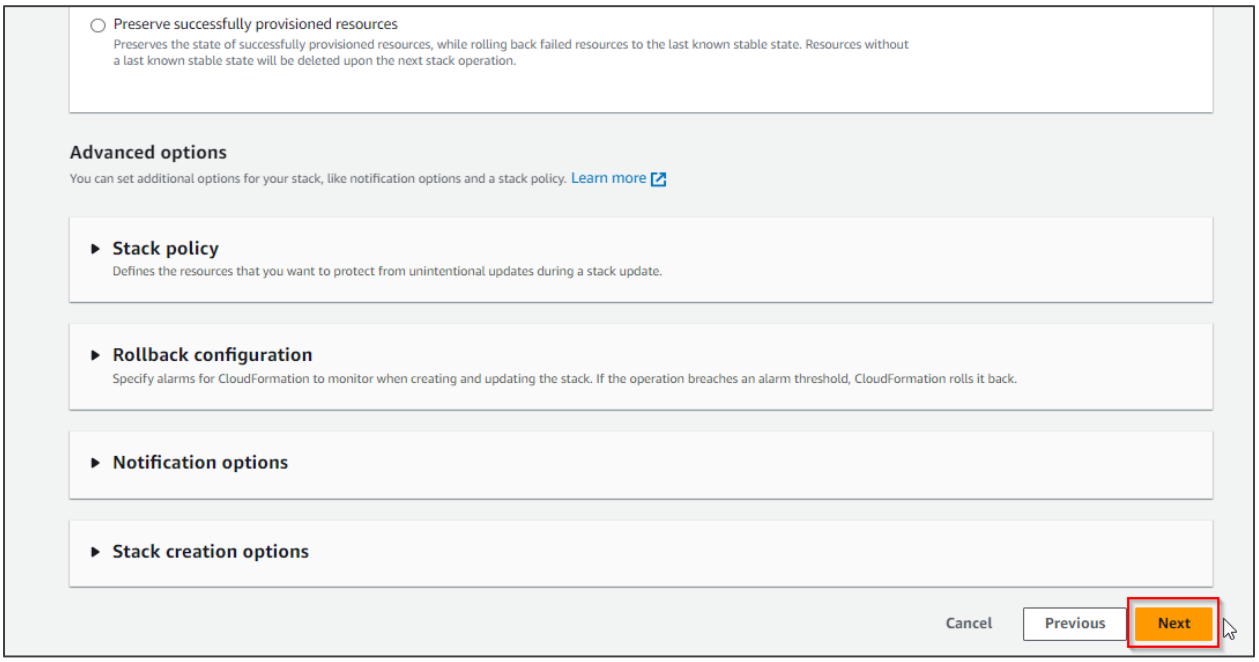
1.12 Add the **Stack name**



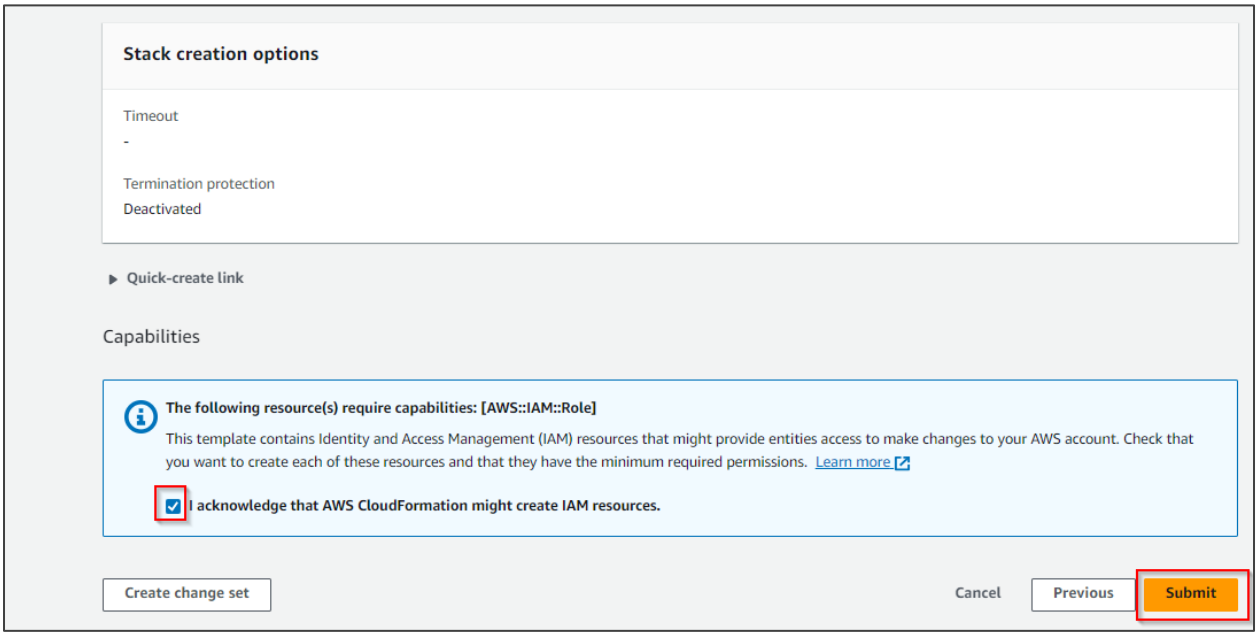
1.13 Click on the **Next** button



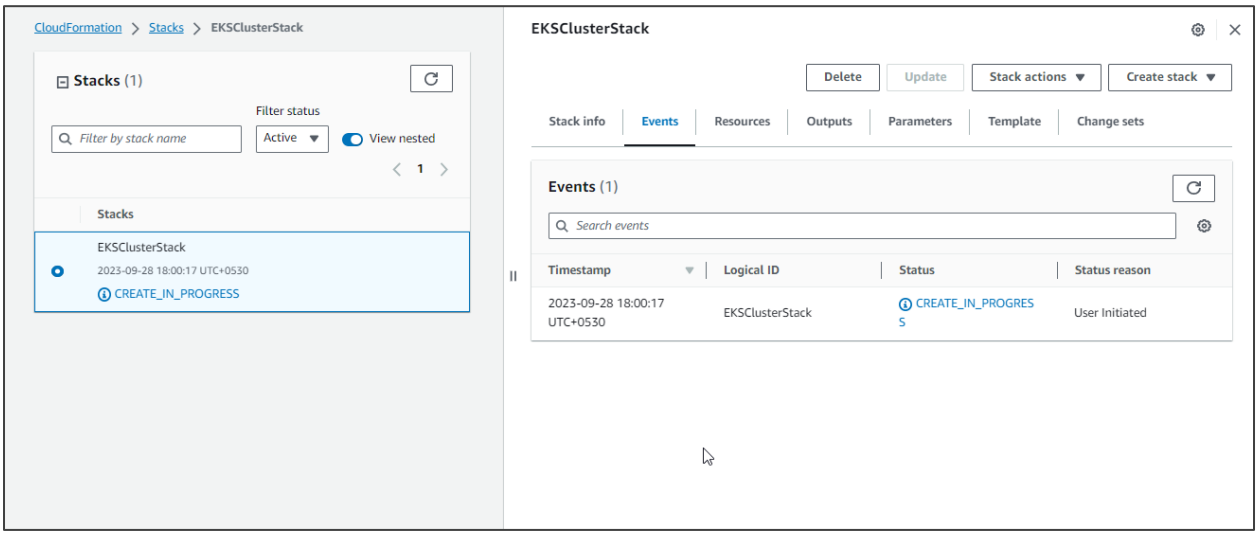
1.14 Click on the **Next** button



1.15 Click on the checkbox to acknowledge the changes and then click on the **Submit** button

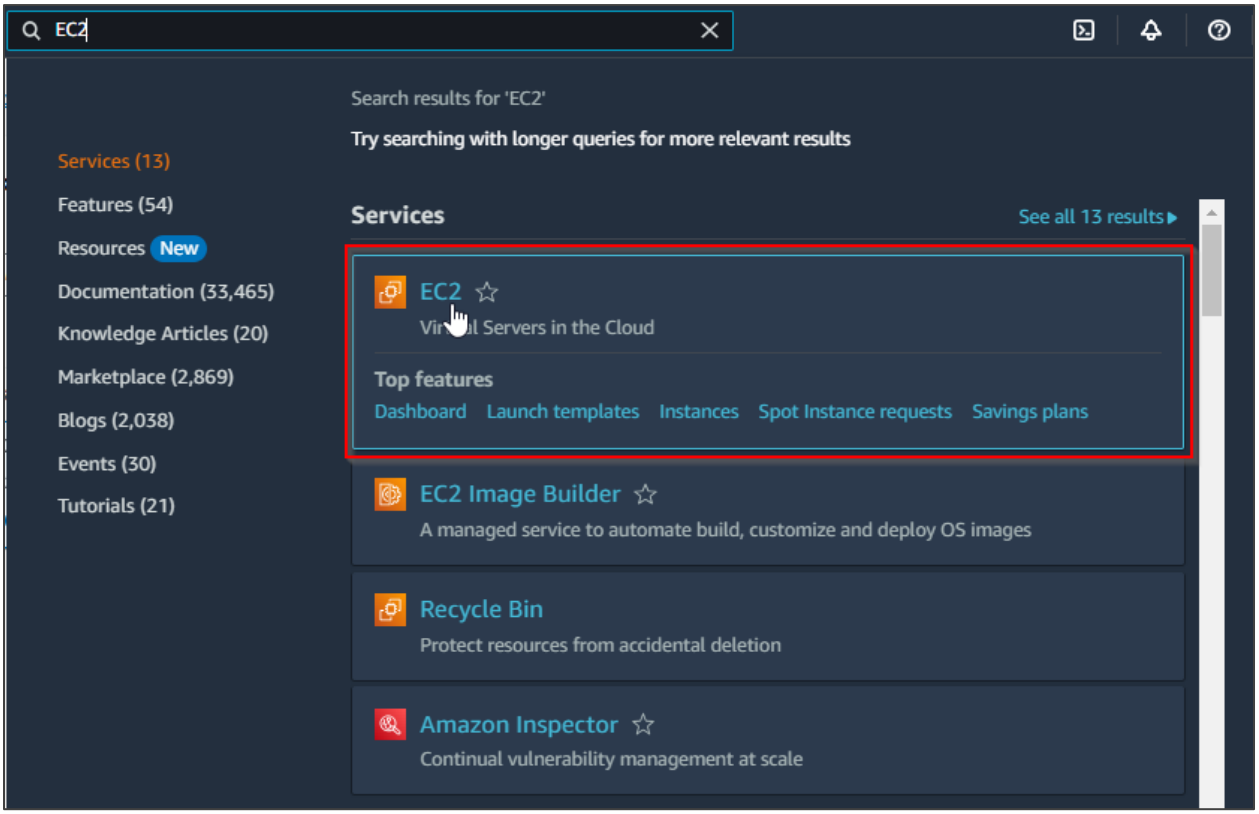


You will see the following interface:

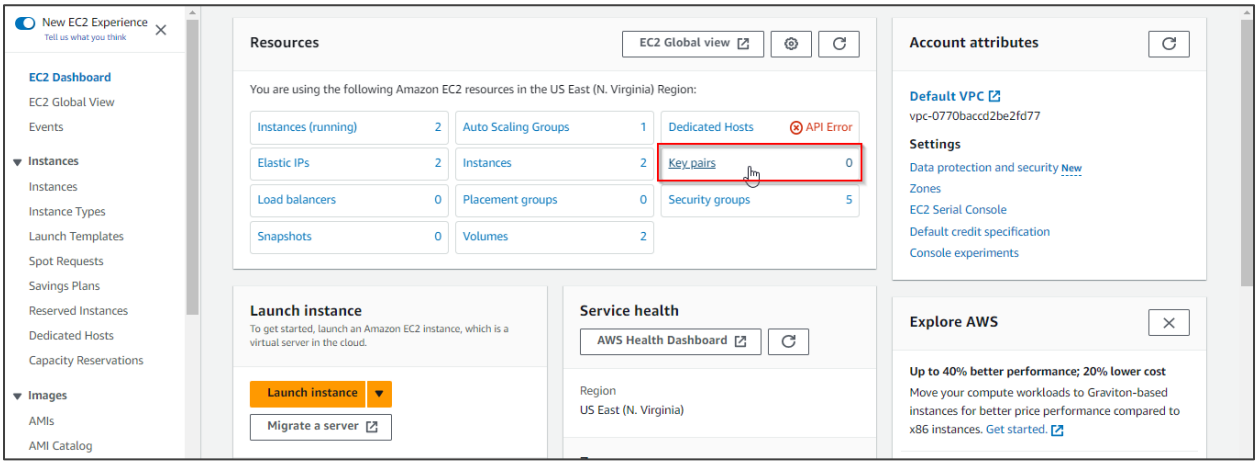


Step 2: Deploy and validate the EKS Cluster

2.1 Search for and click on **EC2**

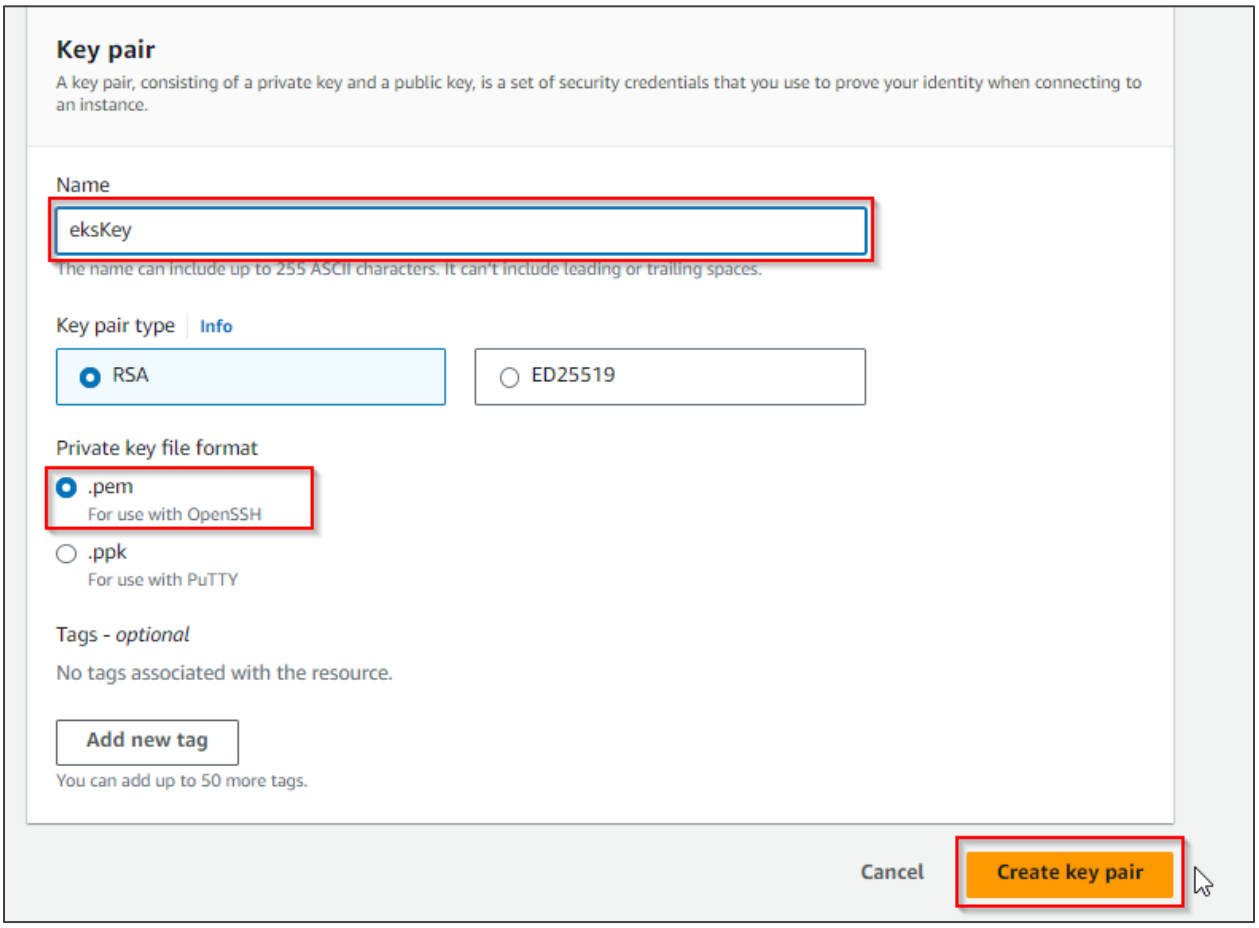


2.2 Click on **Key pairs**

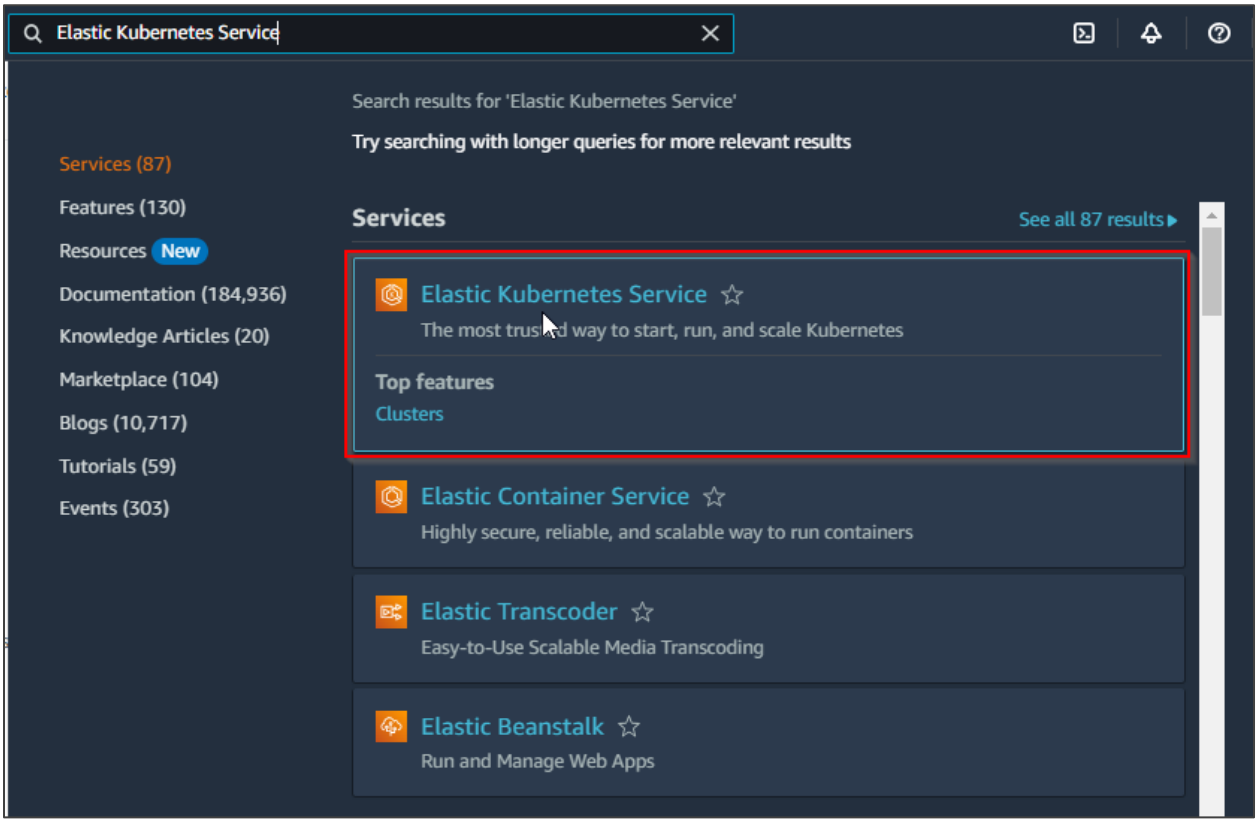


2.3 Click on the **Create key pair** button

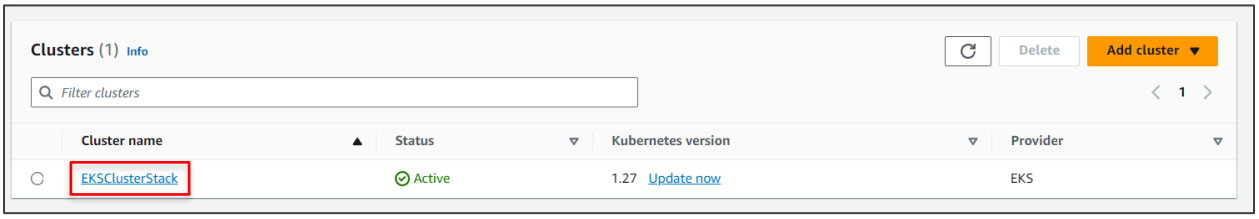
2.4 Add the **name** of the key, select **.pem** as **Private key file format**, and click on the **Create key pair** button



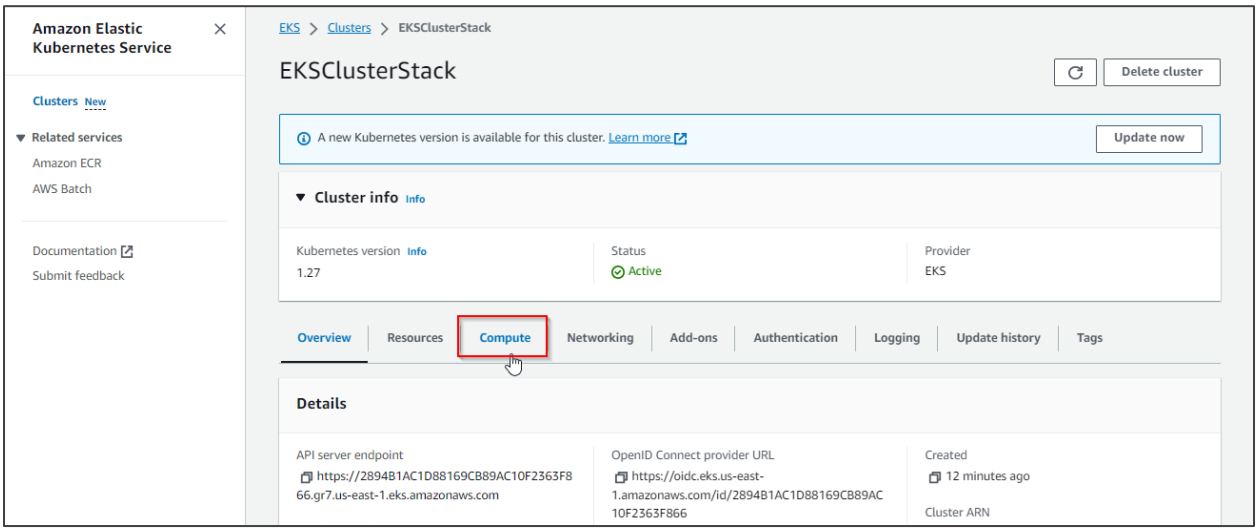
2.5 Once the key pair is created, search for and click on the **Elastic Kubernetes Service** to validate if the EKS cluster is launched



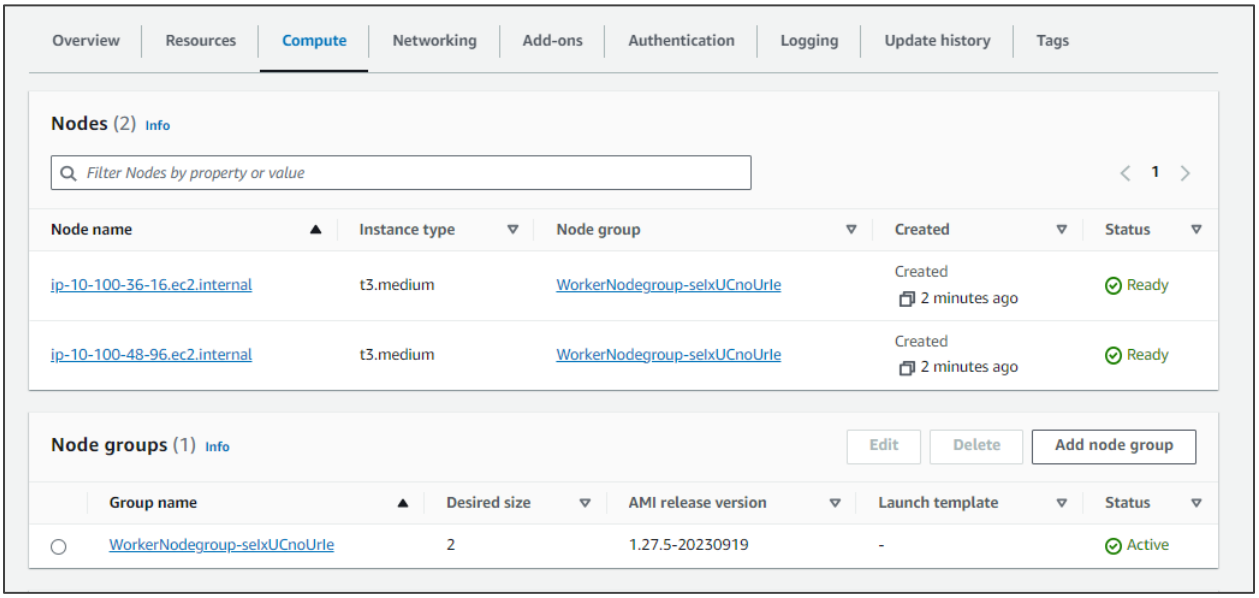
2.6 Click on the **name** of the cluster



2.7 Click on **Compute**



You will see the following interface:



By following these steps, you have successfully automated the provisioning of cloud resources using the CloudFormation service, creating a tailored template for the swift deployment of new EKS clusters.